

Canonical Labelling and Graph Isomorphism: A C++ Reimplementation of Nauty

Auteur: Rodolphe Prévot

Promoteur: Robin Petit

Directeur: Gwenaël Joret

Université libre de Bruxelles
Département d'Informatique

Septembre 2026

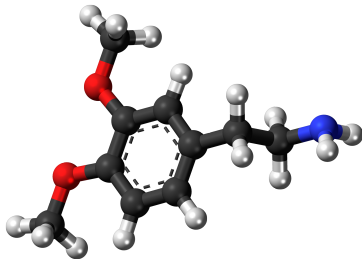
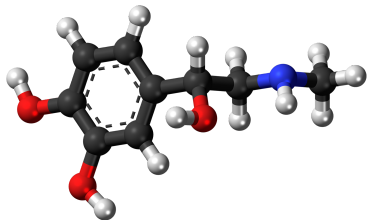


- 1 Contexte
- 2 Nauty
- 3 Conclusion
- 4 Annexe

Contexte



Isomorphisme



Structurellement les mêmes?

Exemple (1/2)

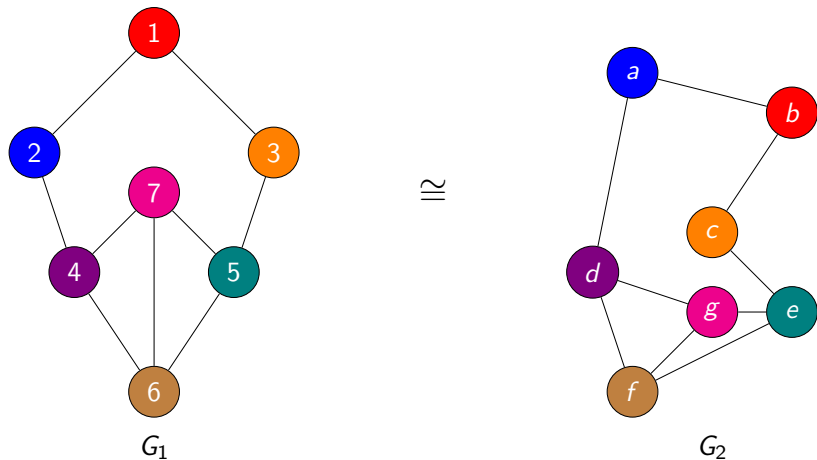


Figure 1: Isomorphisme

Exemple (2/2)

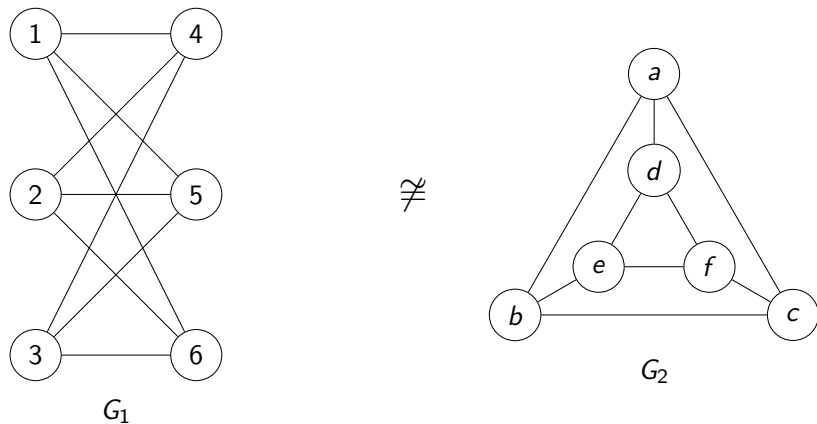
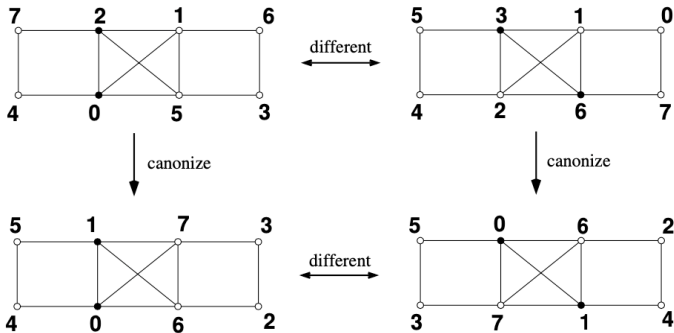


Figure 2: Pas d'isomorphisme

- Problème d'isomorphisme de graphe (IG)
- Complexité: $IG \in NP$, $IG \stackrel{?}{\in} P$, $IG \stackrel{?}{\in} NPC$
- Considéré NPI
- Meilleur algorithme quasi-polynomial: László Babai (2016)
- Forme canonique plus efficace en pratique (*Nauty*)

Forme canonique

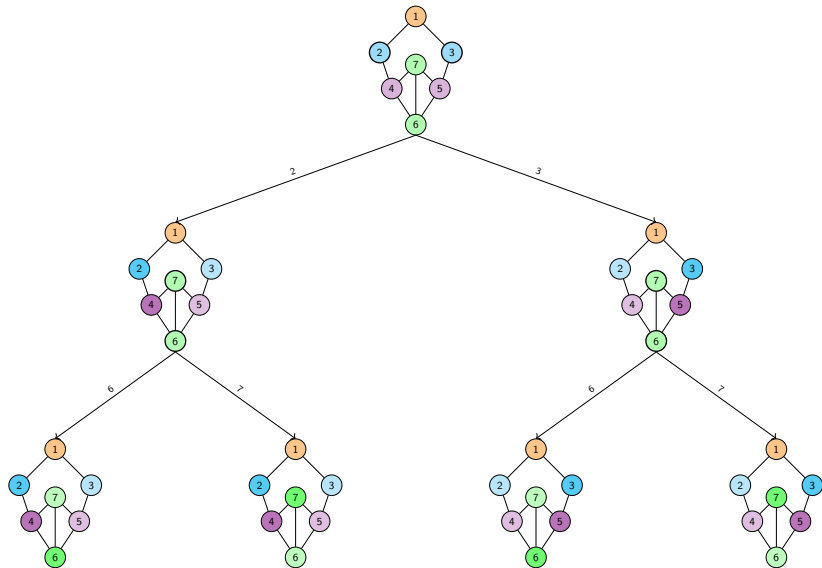
Principe: Problème d'équivalence en problème d'égalité



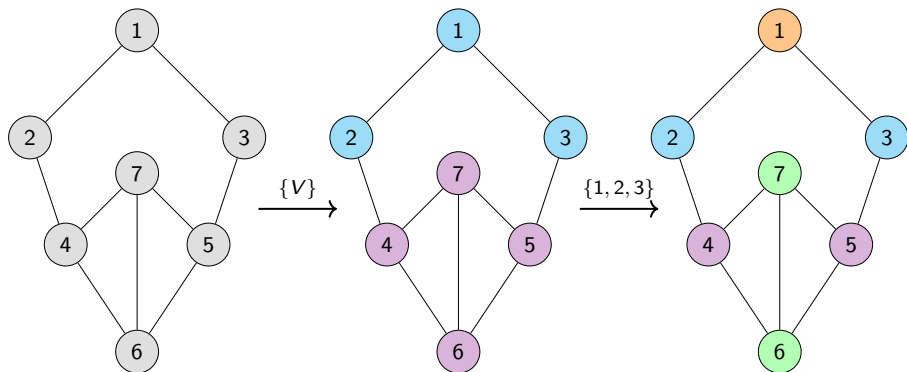
Nauty



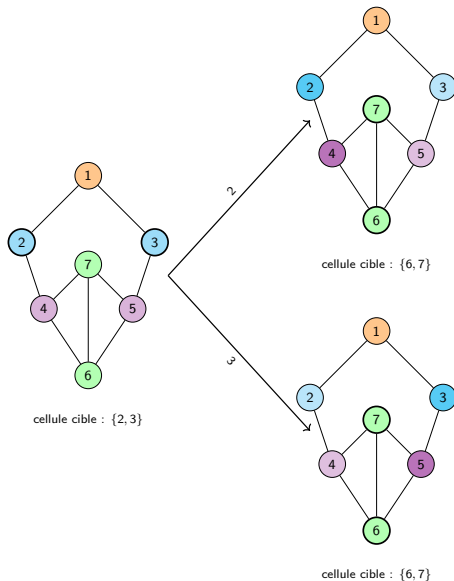
Arbre de recherche



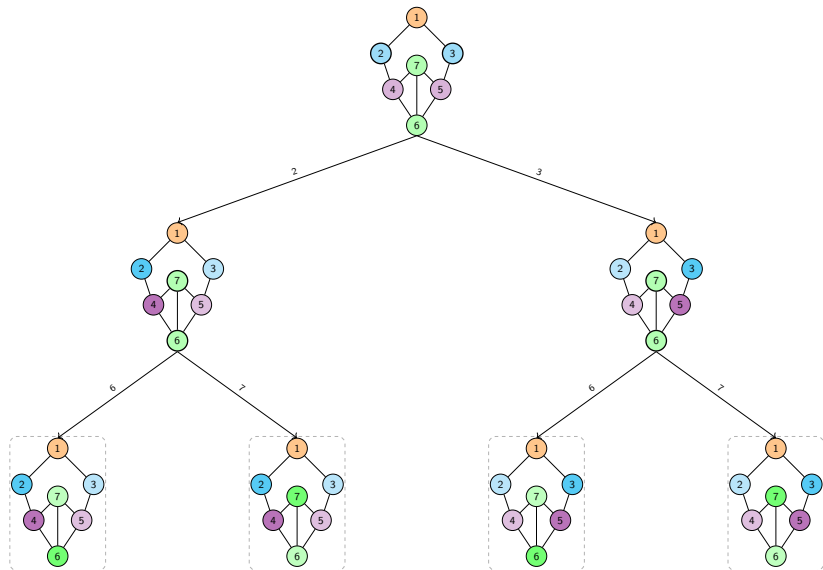
Fonction: raffinement



Fonction: sélection classe



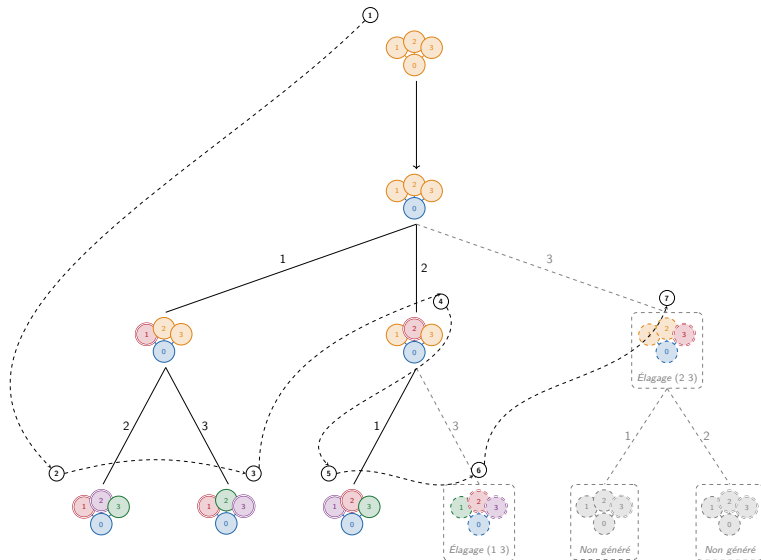
Fonction: invariant + forme canonique



Stratégies d'élagages

- 2 stratégies basées sur l'invariant
- 1 stratégie basée sur les automorphismes
- Élagage par automorphisme plus performant
- Groupe d'automorphisme en cadeau 🎁

Élagage par automorphisme



Conclusion



- *Nauty*: incontournable pour résoudre IG
- Fonctionnalités rajoutées entre temps
- Implémentation moins bonne mais respecte bien la logique

Merci pour votre attention!

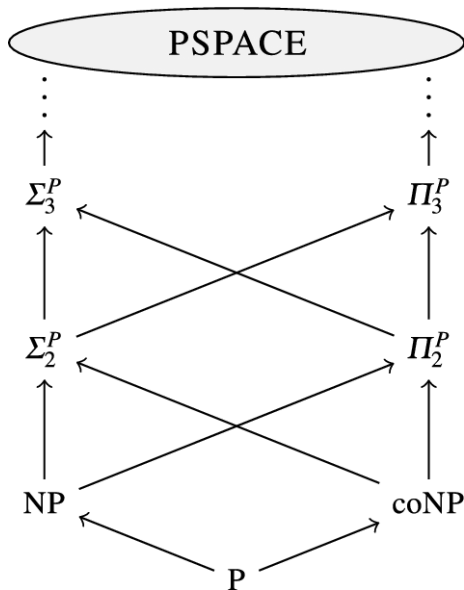
Questions?

rodolphe.prevot@ulb.be

Annexe



Polynomial-Time Hierarchy



- Probablement pas NPC car ETH
- avant 2016: meilleur algorithme en $e^{\mathcal{O}(\sqrt{n \log n})}$
- Algorithme quasi-polynomial, purement théorique pour l'instant
- appartient à SPP
- Différentes complexités en fonction de la famille de graphe

Colour refinement (1/2)

- 1: **Input:** A graph $G = (V, E)$
- 2: **Initialisation:** Assign the same colour to every vertex.
- 3: **repeat**
- 4: **Refinement Step:**
- 5: **for all** vertices v, w with the same colour **do**
- 6: **if** v and w have different numbers of neighbours in some colour class **then**
- 7: Assign different colours to v and w .
- 8: **end if**
- 9: **end for**
- 10: **until** the colouring is stable
- 11: **Return:** the stable colouring.

- *Nauty* se base dessus mais avec des différences
- Identifie un graphe G s'il permet de le distinguer de tous les graphes non isomorphes à G
- Nécessaire mais pas suffisante

Différents types d'invariants

- Nombre de triangle par couleur: $O(n \cdot d^2)$
- 2-chemin depuis une couleur: $O(n + m)$
- Triple par couleur: $O(n^4)$
- Invariant sommet par *Nauty*
- Changement d'invariant à faire

Stratégie basée sur l'invariant

$\mathcal{P}_A$

Éliminer les branches dont l'invariant ne peut pas atteindre la valeur maximale: à chaque niveau de l'arbre, on conserve le meilleur invariant trouvé jusqu'à présent, et tout autre nœud situé au même niveau dont l'invariant est strictement inférieur voit sa branche élaguée.

$\mathcal{P}_B$

Les noeuds sont comparés à un nœud de référence (noeud sur le chemin de la première feuille rencontrée). Si les deux invariants diffèrent, le sous-arbre courant est élagué. L'objectif n'est pas réellement de trouver le meilleur invariant, mais plutôt de trouver des nouveaux automorphismes.

En pratique

Il faut que les 2 conditions soient combinées pour élaguer le noeud.

Stratégie basée sur l'automorphisme

$\mathcal{P}_C$

Exploiter les automorphismes déjà découverts: si un automorphisme g associe le nœud courant σ à un nœud σ^g déjà exploré, alors les deux sous-arbres sont isomorphes. L'exploration de σ^g étant redondante, son sous-arbre est élagué.

Utilité supplémentaire

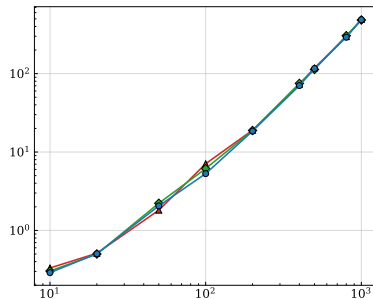
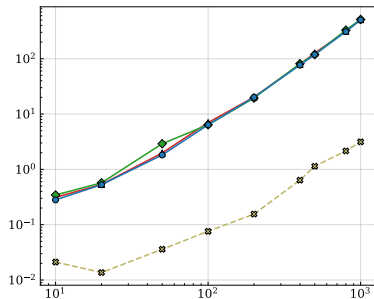
Cette stratégie permet d'avoir le groupe d'automorphisme. L'algorithme donne 2 choses: la forme canonique + groupe d'automorphisme au passage.

- Utilisé pour $\mathcal{P}_C$
- Garde le groupe d'automorphisme
- Complexité en $O(n^2 \log^3 |G| + tn \log |G|)$
- Existe une version randomisée: $O(n^3 \log^c n)$

- Plusieurs experimentation faites
- Temps d'exécution: avec élagage vs sans
- 6 types de graphes
- Performance sur l'élagage
- Légende graphiques ci-dessous
- abscisse = nombre de sommets; ordonné = temps en ms

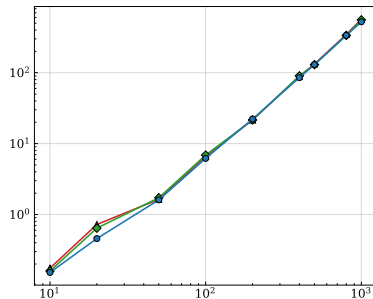
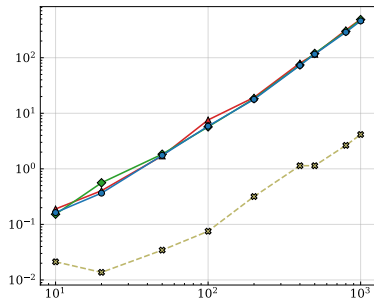
▲ triangle ◆ paths ● triple ✕ *Nauty*

Temps d'exécution (1/6)



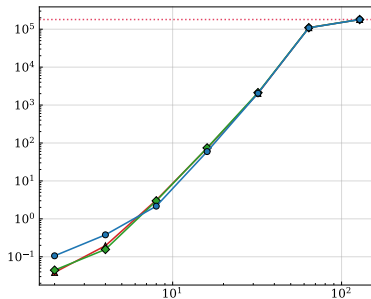
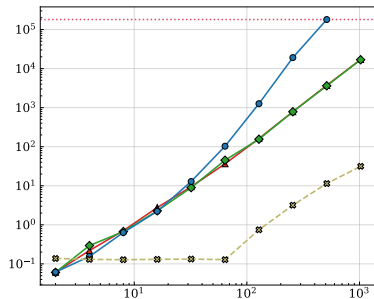
Biparti

Temps d'exécution (2/6)



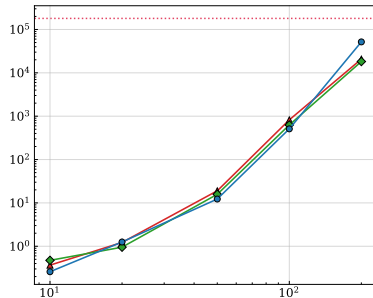
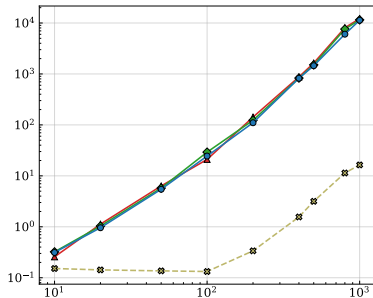
graphe aléatoire $\mathcal{G}(n, 1/2)$

Temps d'exécution (3/6)



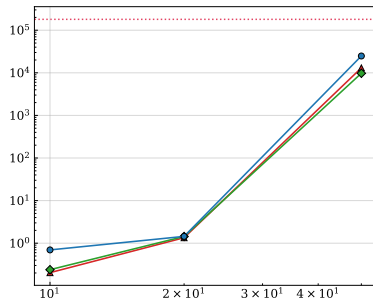
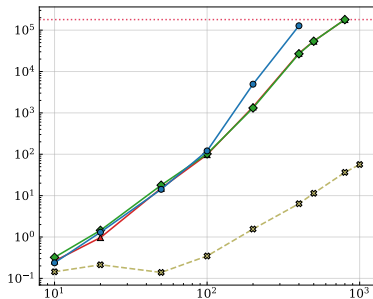
Hypercube

Temps d'exécution (4/6)



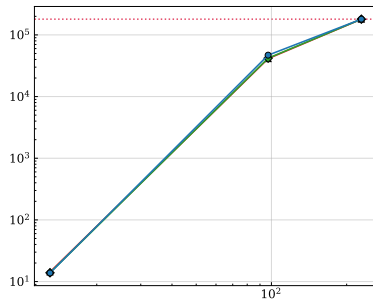
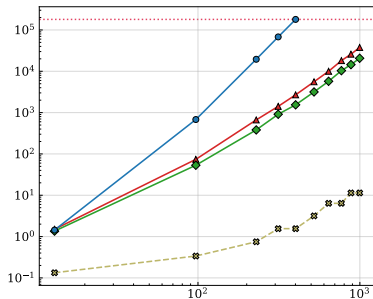
Arbres

Temps d'exécution (5/6)



Biparti

Temps d'exécution (6/6)



Paley

